Milestone 2 : week 3 : Feature extraction

1. Overview

Your pipeline is designed to **extract discriminative features from scanned images** for tasks such as **scanner identification** or **image forensics**. The general steps are:

1. Preprocess images (grayscale conversion, resizing).
2. Extract multiple feature types:
    - **PRNU noise (sensor fingerprint)**
    - **FFT features (frequency domain analysis)**
    - **LBP (texture features)**
    - **Edge features (Sobel gradients)**
3. Combine features into a single feature vector per image.
4. Save all features to a CSV for machine learning or analysis.
5. Visualize sample images per class to ensure dataset correctness.

---

2. Detailed Feature Extraction Methods

2.1 PRNU (Photo-Response Non-Uniformity) Noise Extraction

Code: `extract_prnu_noise(img)`

Theory:

- **PRNU** is a unique sensor pattern noise caused by slight manufacturing imperfections in camera or scanner sensors.
- Each scanner leaves a **consistent noise fingerprint** on every image it scans.
- This is crucial for **forensic analysis**, allowing identification of the scanner that produced the image.

Steps in Code:
Convert image to float in [0,1] range for precision:

```
img = np.float32(img) / 255.0
```

1.

Apply **wavelet decomposition**:

```
coeffs = pywt.wavedec2(img, "sym4", level=3)
```

2.
    - Decomposes image into **low-frequency (approximation) and high-frequency (detail) components**.
    - PRNU is mainly present in **high-frequency components**, while low-frequency components represent the image content.

Zero out approximation coefficients to remove image content:

```
coeffs_noise[0] *= 0
```

3.
4. Reconstruct the image using only high-frequency components → PRNU noise.
5. Convert back to uint8 for further processing.

Key Insight: PRNU is a **scanner fingerprint**, largely independent of image content.

2.2 FFT (Fast Fourier Transform) Features

Code: `extract_fft_features(img)`

Theory:

- **FFT** transforms an image from **spatial domain** to **frequency domain**.
- Useful for detecting **periodic patterns** introduced by scanners, compression artifacts, or repeated structures.

Steps:
Compute 2D FFT:

```
f = np.fft.fft2(img)
```

1.

Shift zero frequency to the center:

```
fshift = np.fft.fftshift(f)
```

2.

Compute magnitude spectrum and logarithm:

```
mag_log = 20 * np.log(np.abs(fshift) + 1)
```

3.
    - Log scaling compresses high dynamic range for numerical stability.
4. Extract **statistical features**:
    - Mean, standard deviation, maximum
    - Percentiles (25th and 75th)
    - Energy (sum of squared magnitudes)
5. Flatten and return as feature vector.

Why FFT?

- Scanners often introduce **regular high-frequency noise**.
- FFT captures **frequency-based discriminative patterns** invisible in the raw spatial domain.

2.3 LBP (Local Binary Patterns) Texture Features

Code: `extract_lbp(img)`

Theory:

- LBP is a **texture descriptor** capturing **micro-patterns** in an image.
- Each pixel is compared to its neighbors:
    - Neighbor ≥ center → 1
    - Neighbor < center → 0
- Encodes local structures like **edges, spots, flat regions** into a **binary code**.

Steps:
Compute LBP:

```
lbp = local_binary_pattern(img, P=8, R=1, method="uniform")
```

1.
    - `P=8` → 8 neighbors
    - `R=1` → radius = 1 pixel

- `uniform` → reduces code complexity (common patterns)

Build histogram of LBP codes:

```
hist, _ = np.histogram(lbp.ravel(), bins=20, range=(0,20))
```

2.
3. Normalize histogram to unit sum for **scale invariance**.

Why LBP?

- Captures **local texture variations** caused by different scanner hardware or optics.
- Works well with grayscale images and is **rotation-invariant** with proper settings.

2.4 Edge Features (Sobel Gradients)

Code: `extract_edges(img)`

Theory:

- Edges are regions of rapid intensity change.
- **Sobel operator** calculates approximate gradients along x and y axes:
  Gx=∂I∂x,Gy=∂I∂yG_x = \frac{\partial I}{\partial x}, \quad G_y = \frac{\partial I}{\partial y}Gx=∂x∂I,Gy=∂y∂I
- Gradient magnitude:
  M=Gx2+Gy2M = \sqrt{G_x^2 + G_y^2}M=Gx2+Gy2

Steps:
Compute Sobel derivatives:

```
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0)
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1)
```

1.
2. Compute gradient magnitude.
3. Extract **mean, standard deviation, max** as edge features.

Why Edges?

- Edge statistics vary subtly based on **scanner optics, resolution, and interpolation artifacts**.

2.5 Combined Feature Vector

Code: `extract_features(img_path)`

Features concatenated into a single vector:

```
[FFT features (6), LBP histogram (20), Edge features (3)] → 29 features
```

-
- These features collectively capture:
    - **Sensor-level noise** (PRNU + FFT)
    - **Texture characteristics** (LBP)
    - **Structural properties** (edges)

This vector is ready for **machine learning classifiers** (SVM, Random Forest, Neural Networks) for **scanner classification**.

---

3. Dataset Processing

Code: `for root, dirs, files in os.walk(dataset_root): ...`

Traverses dataset folder structure:

```
dataset_root/
    Scanner_A/
    Scanner_B/
    ...
```

-
- For each image:
    - Extract features.
    - Assign label = folder name.
- Save all features to **CSV** for later analysis.

CSV Structure:

```
image_path,label,fft_0,...,fft_5,lbp_0,...,lbp_19,edge_0,...,edge_2
```

---

4. Dataset Visualization

Code:

```
for i, cls in enumerate(classes):
    path = df[df["label"] == cls].iloc[0]["image_path"]
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
```

Purpose:

- Confirm that images are **correctly labeled**.
- Quick visual sanity check before training models.
- Plot **one sample per scanner class**.

Steps:

1. Read first image of each class.
2. Display using `matplotlib.pyplot`.
3. Show as grayscale, remove axes, add titles.

---

5. Practical Notes

1. **Resizing to 512×512** ensures uniform feature dimensions across images.
2. **Grayscale conversion** reduces complexity and focuses on structural + noise features.
3. **Wavelet-based PRNU** is robust to image content variations.
4. **FFT + LBP + Edge combination** gives a **rich feature set** capturing scanner-specific characteristics.
5. The **CSV output** is suitable for:
    - **Machine learning training**
    - **Feature analysis**
    - **Clustering / PCA visualization**

---

6. Applications

- **Scanner identification:** Determine which scanner produced an image.

- **Image forgery detection:** Detect tampering by identifying mismatched PRNU or texture patterns.
- **Forensic analysis:** Trace images back to hardware sources.